

LAB - 4

NAME - DHIRAJ

PRN - 202501040062

DIV - CS1

BATCH - C13

Practical 4

About this unit

Practical 4

Practice Lab Assignment

Unit · 100% completed



Lab Assignment

Unit · 100% completed



Search



ENG
IN



22:18
04-05-2026



4.1.1. Pandas - series creation and manipulation

09:47

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases



Explorer

seriesMa...

Submit

Debugger

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8
9 # Grouping by even and odd numbers and calculating the mean
10 grouped = series.groupby(series % 2 == 0).mean()
11
12 # Display the mean of even and odd numbers with labels
13 grouped.index = ['Even' if is_even else 'Odd' for is_even in
14 grouped.index]
15 print("Mean of even and odd numbers:")
16 print(grouped)
```

Terminal

Test cases

[< Prev](#) [Reset](#) [Submit](#) [Next >](#)

Search



ENG
IN



20:24
04-05-2026



4.1.2. Dictionary to dataframe

43:34

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 new_name = input("New name: ")
18 new_age = int(input("New age: "))
19 df.loc[len(df)] = [new_name, new_age]
20
21 # Display the DataFrame after adding a new row
22 print("After adding a row:\n",df)
23
24 # Modifying a row
25 row_to_modify = int(input("Index of row to modify: "))
```

4.1.3. Student Information

02:50

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases



Explorer

studentin...



Submit

Debugger

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
6 "Grade"])
7
8 # write your code here..
9
10 print("First five rows:")
11 print(data.head())
12
13 print("Average age:", round(data["Age"].mean(), 2))
14
15 print("Students with a grade up to B")
16 print(data[data["Grade"] <= "B"])
```

Terminal

Test cases

< Prev

Reset

Submit

Next >



Search



ENG
IN

20:25
04-05-2026

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 df["Date"] = pd.to_datetime(df["Date"])
11 df["Month"] = df["Date"].dt.to_period("M")
12 df["Sales"] = df["Quantity"] * df["Price"]
13
14 sales = df.groupby("Month")["Sales"].sum()
15 # Find the month with the highest total sales
16 best_month = sales.idxmax()
17 highest_sales = sales.max()
18
19 print(f"Best month: {best_month}")
20 print(f"Total sales: ${highest_sales:.2f}")
21
```

Terminal Test cases

< Prev Reset Submit Next >



Search



ENG
IN

20:25
04-05-2026

4.2.2. Best Selling Product01:00

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFo...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 sales = df.groupby("Product")["Quantity"].sum()
11
12 # Find the product with the highest total quantity sold
13 best_product = sales.idxmax()
14 highest_quantity = sales.max()
15
16 # Display the result
17 print(f"Best selling product: {best_product}")
18 print(f"Total quantity sold: {highest_quantity}")
19
```

Terminal Test cases

4.2.3. City that Sold the Most Products

00:29

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

+

Explorer

monthFo...

Submit

Submit

Debugger

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 best_city = df.groupby("City")["Quantity"].sum().idxmax()
11 # Display the result
12 print(f"City sold the most products: {best_city}")
13
```

Terminal

Test cases

< Prev Reset Submit Next >



Search



ENG
IN

20:25
04-05-2026

4.2.4. Most Frequently Sold Product Pairs

00:47

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Explorer

frequentl...

1 import pandas as pd

2 from itertools import combinations

3 from collections import Counter

4

5 # Prompt user to input the file name

6 file_name = input()

7

8 # Read data from the specified CSV file

9 df = pd.read_csv(file_name)

10

11 # write the code

12

13 grouped = df.groupby('Date')['Product'].apply(list)

14

15 pairs = []

16

17 for products in grouped:

18 if len(products) > 1:

19 products = sorted(products)

20 for i in range(len(products)):

21 for j in range(i+1, len(products)):

22 pairs.append((products[i], products[j]))

23

24 pairs_df = pd.DataFrame(pairs, columns=['Product1', 'Product2'])

25

Terminal Test cases

Submit

Debugger

4.2.5. Titanic Dataset Analysis and Data Cleaning

00:30

Submit

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Pas sen ger d	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bar ked

Sample Data:

Explorer

titanicDat...

Debugger

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. First 5 rows
8 head = data.head()
9 print(head)
10
11 # 2. Last 5 rows
12 tail = data.tail()
13 print(tail)
14
15 # 3. Shape
16 print(data.shape)
17
18 # 4. Full .info() output with memory usage
19 print(data.info())
20
21 # 5. Basic stats
22 print(data.describe())
23
24 # 6. Missing values
25 print(data.isnull().sum())
```

Terminal Test cases

4.2.6. Titanic Dataset Analysis and Data Cleaning - 200:45

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

Pas sen ger Id	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bar ked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
8 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
9
10 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
11 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
12
13 # 3. One-hot encode the 'Embarked' column
14 data = pd.get_dummies(data, columns=['Embarked'], prefix='Embarked',
15 drop_first=True)
16
17 # 4. Get the mean age of passengers
18 mean_age = data['Age'].mean()
19 print(mean_age)
20
21 # 5. Get the median fare of passengers
22 median_fare = data['Fare'].median()
23 print(median_fare)
24
25 # 6. Get the number of passengers by class
```

Terminal Test cases

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Pas sen ger d	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bar ked

Sample Data:

Explorer

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Calculate the survival rate by embarked location
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
16 # 3. Calculate the survival rate by family size
17 print(data.groupby('FamilySize')['Survived'].mean())
18
19 # 4. Calculate the survival rate by being alone
20 print(data.groupby('IsAlone')['Survived'].mean())
21
22 # 5. Get the average fare by class
23 print(data.groupby('Pclass')['Fare'].mean())
24
25 # 6. Get the average age by class
```

Terminal Test cases

Debugger

4.2.8. Titanic Dataset Analysis and Data Cleaning - 400:50

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Pas sen ger Id	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bar ked

Sample Data:

titanicDat...Submit

Explorer

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 # 1. Get the number of survivors by gender
10 print(data[data['Survived'] == 1]['Sex'].value_counts())
11
12 # 2. Get the number of non-survivors by gender
13 print(data[data['Survived'] == 0]['Sex'].value_counts())
14
15 # 3. Get the number of survivors by embarked location
16 print(data[data['Survived'] == 1]['Embarked_S'].value_counts())
17
18 # 4. Get the number of non-survivors by embarked location
19 print(data[data['Survived'] == 0]['Embarked_S'].value_counts())
20
21 # 5. Calculate the percentage of children (Age < 18) who survived
22 children = data[data['Age'] < 18]
23 print(children['Survived'].mean())
24 # 6. Calculate the percentage of adults (Age >= 18) who survived
25 adults = data[data['Age'] >= 18]
```

Debugger

Terminal

Test cases